

TEST MEMORIA (ASISTENTE C)

Perfecto. Lo ideal es probar tres capacidades distintas:

1. Memoria episódica
→ recordar experiencias pasadas
2. Memoria semántica/LTM
→ consolidar conocimiento reusable
3. Razonamiento + retrieval
→ usar memoria para decidir mejor

Te propongo tests “tipo benchmark” que luego puedes automatizar.

TEST 1 — MEMORIA EPISÓDICA

“Recordar errores previos”

Objetivo:

Verificar que el sistema:

- guarda una experiencia
- la recupera después
- evita repetir el error

INPUT — SESIÓN

1

Usuario:

txt

Estoy desplegando Next.js + Prisma en Vercel y obtengo este error:

PrismaClientInitializationError:

Prisma Client could not locate the Query Engine for runtime "rhel-openssl"

¿Cómo lo soluciono?

Respuesta esperada del agente:

txt

```
- Detecta problema de runtime/binarios
- Sugiere:
  prisma generate
  binaryTargets

runtime=nodejs
```

Luego el sistema debe guardar:

JSON

```
{
  "type": "episode",
  "tags": ["prisma", "vercel", "deployment"],
  "problem": "Prisma query engine missing on Vercel",
  "solution": "Use runtime=nodejs and regenerate prisma client",
  "importance": 0.9
}
```

INPUT — SESIÓN 2

Horas después:

Usuario:

txt

Tengo otro deploy en Vercel con Prisma y falla en producción.

ÉXITO ESPERADO

El agente debe:

- recuperar automáticamente la memoria previa
- mencionar el patrón ANTES de que el usuario explique todo
- usar lenguaje tipo:

txt

En experiencias previas con Prisma + Vercel:

- el problema estaba relacionado con runtime/binarios

- revisaría runtime=nodejs y prisma generate primero

VALIDACIÓN

PASS si:

- ✓ recupera memoria relevante
- ✓ menciona solución previa
- ✓ contextualiza correctamente

FAIL si:

- ✗ responde genérico
- ✗ no usa experiencia previa

TEST 2 — CONSOLIDACIÓN DE CONOCIMIENTO

“Convertir experiencias en patrones”

Objetivo:

Verificar que múltiples episodios generan conocimiento abstracto.

INPUTS

Simular 3 memorias:

JSON

```
[
  {
    "problem": "Prisma fails on Vercel Edge runtime"
  },
  {
    "problem": "Drizzle ORM incompatible with edge runtime"
  },
  {
    "problem": "Node APIs unavailable in edge deployment"
  }
]
```

EJECUTAR

txt

memory_consolidation()

RESULTADO ESPERADO

El sistema genera:

JSON

```
{  
  "type": "pattern",  
  "text": "ORMs and Node-dependent libraries often fail in Edge runtimes",  
  "confidence": 0.91,  
  "derived_from": ["mem1", "mem2", "mem3"]  
}
```

NUEVA CONSULTA

Usuario:

txt

¿Puedo usar Prisma en Edge Runtime?

RESPUESTA ESPERADA

El agente:

- NO responde desde memoria literal
- responde desde conocimiento consolidado

Ejemplo:

txt

Hay un patrón recurrente:
muchos ORMs que dependen de APIs Node presentan problemas en Edge runtimes.

Prisma puede funcionar parcialmente, pero normalmente se recomienda:

- Node runtime

- Prisma Accelerate/Data Proxy

VALIDACIÓN

PASS si:

- ✓ sintetiza patrón
- ✓ abstrae correctamente
- ✓ usa consolidación

FAIL si:

- ✗ solo concatena memorias
- ✗ no generaliza

TEST 3 — RAZONAMIENTO + FACT CHECK

“Usar memoria sin alucinar”

Objetivo:

Verificar:

- retrieval
- razonamiento
- confidence scoring
- fact checking

MEMORIA PREVIA

JSON

```
{  
  "type": "fact",  
  "text": "Railway suspends inactive PostgreSQL databases on free tier",  
  "confidence": 0.85  
}
```

INPUT

Usuario:

txt

Mi app pierde conexión a PostgreSQL después de varias horas sin uso.
Uso Railway.

¿Qué podría estar pasando?

COMPORTAMIENTOS ESPERADOS

El sistema debe:

1. Recuperar memoria relevante
2. Generar hipótesis
3. Marcar incertidumbre apropiadamente

RESPUESTA IDEAL

txt

Una posible causa es la suspensión automática de bases PostgreSQL inactivas en Railway free tier.

Tengo memoria previa relacionada con ese comportamiento, aunque conviene verificar:

- tu plan actual
- logs de Railway
- timeouts de conexión

Confianza estimada: media-alta.

VALIDACIÓN

PASS si:

- ✓ usa memoria relevante
- ✓ expresa incertidumbre
- ✓ no afirma falsedades absolutas
- ✓ propone verificación

FAIL si:

- ✗ inventa causas
- ✗ responde con exceso de confianza
- ✗ ignora memoria

BONUS — TEST AVANZADO

“Aprendizaje real”

Objetivo:

Ver si el sistema mejora después de varias iteraciones.

Proceso:

- repetir mismo tipo de problema 5 veces
- medir:
 - velocidad
 - precisión
 - retrieval score
 - tokens usados
 - necesidad de aclaraciones

Éxito:

- menos contexto necesario
- respuestas más precisas
- recuperación más rápida

ESTRUCTURA SUGERIDA

txt

```
tests/
├── memory/
│   ├── episodic.test.json
│   ├── consolidation.test.json
│   └── reasoning.test.json
├── fixtures/
│   └── memories.json
└── expected/
    └── outputs.json
```

MÉTRICAS IMPORTANTES

Te recomiendo medir:

JSON

```
{
  "retrieval_relevance": 0.91,
  "memory_used": true,
  "hallucination_score": 0.12,
  "reasoning_depth": 0.84,
  "confidence_alignment": 0.89
}
```

Eso te permitirá evaluar si realmente “aprende”.

SIGUIENTE PASO RECOMENDADO

Ahora lo ideal sería definir:

1. Formato exacto de MemoryRecord
2. Pipeline:
ingest → embed → retrieve → reflect → consolidate
3. Política:
qué guardar y qué ignorar
4. Estrategia de scoring:
similarity + importance + recency

Y después construir:

txt

MemoryEngine